3er entrega (parte 1) Calidad, verificación, validación y pruebas de software

Calidad del software

La calidad del software es uno de los principales objetivos de la Ingeniería de Software. Un sistema no puede considerarse satisfactorio únicamente porque sus funcionalidades principales se ejecuten correctamente. También debe cumplir los requisitos establecidos, responder adecuadamente ante situaciones excepcionales y presentar características que permitan utilizarlo y mantenerlo de manera confiable.

Según Pressman, la calidad del software está relacionada con la aplicación de procesos adecuados, la construcción de productos que satisfagan los requisitos y el cumplimiento de atributos de calidad relevantes para los usuarios y demás interesados.

Sommerville (2016) señala que la calidad no depende exclusivamente de la corrección funcional. También deben considerarse propiedades como la confiabilidad, la seguridad, la eficiencia y la facilidad de mantenimiento.

En consecuencia, la evaluación de la calidad requiere considerar tanto los aspectos funcionales como los no funcionales del sistema.

Características de calidad

- Funcionalidad: el sistema realiza correctamente las operaciones requeridas.
- Confiabilidad: funciona de manera consistente ante las condiciones previstas.
- Usabilidad: los usuarios pueden comprender y utilizar sus funcionalidades.
- Eficiencia: utiliza adecuadamente los recursos disponibles y mantiene tiempos de respuesta aceptables.
- Seguridad: protege los datos y restringe el acceso a las funciones según los permisos establecidos.
- Mantenibilidad: permite introducir correcciones y modificaciones sin dificultades innecesarias.

Ejemplo: Sistema de gestión de documentos

Un sistema permite registrar, consultar y modificar documentos. Las operaciones CRUD funcionan correctamente, pero cualquier usuario puede eliminar documentos sin autorización.

Aunque el sistema cumple parte de sus requisitos funcionales, presenta un problema de seguridad que compromete su calidad.

Verificación y validación del software

La verificación y la validación son procesos complementarios utilizados para evaluar el software durante su desarrollo.

Sommerville (2016) distingue entre ambos conceptos a partir de sus objetivos.

Verificación: consiste en comprobar que el producto desarrollado cumple las especificaciones, diseños y requisitos establecidos.

Validación: consiste en comprobar que el producto satisface las necesidades reales de sus usuarios y resulta adecuado para el propósito previsto.

Estas actividades deben realizarse durante el desarrollo y no solamente al finalizarlo.

Diferencias entre verificación y validación

Verificación	Validación
Comprueba el cumplimiento de las especificaciones.	Comprueba la adecuación a las necesidades del usuario.
Se relaciona con construir correctamente el producto especificado.	Se relaciona con construir el producto que realmente se necesita.
Puede incluir revisiones, inspecciones y pruebas.	Puede incluir pruebas funcionales, pruebas de aceptación y evaluaciones de usuarios.
Se compara el producto con requisitos o modelos.	Se compara el producto con las necesidades y expectativas de uso.

Ejemplo: Sistema de gestión de documentos

El requerimiento RF-05 establece que el sistema debe permitir buscar documentos por número de identificación.

- Verificación: comprobar que la búsqueda funciona de acuerdo con lo establecido en RF-05.
- Validación: solicitar **al usuario** que utilice la búsqueda en una situación real y comprobar que le permite encontrar la documentación que necesita.

Un sistema puede superar la verificación y, sin embargo, no satisfacer las necesidades de sus usuarios si los requisitos originales fueron incompletos o incorrectos.

Pruebas de software

Las pruebas de software son actividades destinadas a ejecutar y evaluar un programa para identificar defectos y comprobar su comportamiento frente a condiciones determinadas.

Pressman presenta las pruebas como parte fundamental de la estrategia de **aseguramiento de la calidad**. Su finalidad comprende tanto descubrir errores como aportar evidencias acerca del cumplimiento de las especificaciones.

Sommerville (2016) distingue dos objetivos importantes de las pruebas:

1. Pruebas de validación: demostrar, mediante casos seleccionados, que el sistema cumple las funcionalidades previstas.
2. Pruebas de defectos: encontrar situaciones en las que el sistema presenta comportamientos incorrectos.

Las pruebas permiten obtener evidencia sobre la calidad, pero no garantizan que el software esté completamente libre de errores.

Niveles de prueba

Pruebas unitarias: evalúan individualmente unidades pequeñas del software, como funciones, métodos o clases. Generalmente las realiza el programador.

Pruebas de integración: evalúan la interacción entre componentes que funcionan conjuntamente.

Pruebas de sistema: comprueban el comportamiento del sistema integrado y completo.

Pruebas de aceptación: permiten determinar si el sistema satisface las condiciones necesarias para ser aceptado por el cliente o usuario.

Ejemplo: Sistema de gestión de usuarios

Nivel	Prueba
Unitaria	Verificar que el método de validación de contraseñas identifica entradas inválidas.
Integración	Comprobar que el formulario de registro se comunica correctamente con el backend y la base de datos.
Sistema	Comprobar el proceso completo de registro, autenticación y acceso a funcionalidades.
Aceptación	Solicitar al cliente que realice las operaciones habituales y determine si cumplen lo acordado.

Pruebas funcionales

Las pruebas funcionales comprueban si un sistema realiza las funciones establecidas en sus requisitos.

Generalmente utilizan un enfoque de caja negra, en el cual se evalúa el comportamiento observable del software sin necesidad de conocer su implementación interna.

Pressman explica que las técnicas de pruebas basadas en especificaciones permiten elaborar casos utilizando entradas, salidas esperadas y condiciones de funcionamiento.

Para construir una prueba funcional es necesario identificar:

- El requerimiento que se evaluará.
- Las condiciones iniciales.
- Los datos de entrada.
- La operación que debe realizarse.
- El resultado esperado.
- El resultado obtenido.
- El estado de la prueba.

Ejemplo: Inicio de sesión

Requerimiento RF-03: el sistema debe permitir el acceso de usuarios registrados mediante sus credenciales.

Entrada	Resultado esperado
Usuario y contraseña correctos	Acceso autorizado.
Usuario correcto y contraseña incorrecta	Acceso rechazado.
Usuario inexistente	Acceso rechazado.
Campos vacíos	Mensaje de validación.

El objetivo no es solamente comprobar que el inicio de sesión funciona con credenciales correctas, sino también verificar cómo responde ante datos incorrectos o incompletos.

Diseño de casos de prueba

Un caso de prueba es una especificación que establece las condiciones, entradas, acciones y resultados esperados para evaluar un comportamiento determinado del software.

Según Sommerville (2016), los casos de prueba deben seleccionarse de forma que permitan comprobar el comportamiento normal del sistema y detectar posibles defectos.

Pressman y Maxim (2020) destacan la importancia del diseño sistemático de las pruebas, evitando depender únicamente de ejecuciones improvisadas.

Estructura de un caso de prueba

Campo	Descripción
Identificador	Código único del caso de prueba.
Requerimiento asociado	RF o RNF que se pretende evaluar.
Nombre	Funcionalidad o comportamiento evaluado.
Precondiciones	Situación inicial necesaria para ejecutar la prueba.
Datos de entrada	Valores utilizados durante la ejecución.
Procedimiento	Pasos necesarios para realizar la prueba.
Resultado esperado	Comportamiento correcto del sistema.
Resultado obtenido	Comportamiento observado durante la ejecución.
Estado	Aprobado, fallido o bloqueado.
Evidencia	Captura, registro u otro elemento que permita comprobar el resultado.

Ejemplo: Caso de prueba CP-01

Nombre: Registro de un usuario con datos válidos.

Requerimiento: RF-01, registrar usuarios.

Precondiciones: el usuario administrador inició sesión correctamente.

Datos de entrada:

- Nombre: Ana Pérez.
- Correo electrónico: ana@ejemplo.com.
- Contraseña: Segura2026!
- Rol: Operador.

Procedimiento:

1. Acceder a la opción de administración de usuarios.
2. Seleccionar «Nuevo usuario».
3. Completar los campos requeridos.
4. Presionar «Guardar».
5. Consultar el listado de usuarios.

Resultado esperado: el sistema registra correctamente al usuario y lo muestra en el listado.

Resultado obtenido: se registra durante la ejecución.

Estado: pendiente de ejecución.

Esta estructura permite mantener un registro verificable de las pruebas realizadas.

Trazabilidad entre requerimientos y pruebas

La trazabilidad consiste en mantener relaciones identificables entre los requisitos del sistema y los elementos que permiten implementarlos y verificarlos.

Una matriz de trazabilidad facilita comprobar que cada requerimiento cuenta con pruebas apropiadas.

Requerimiento	Caso de uso	Caso de prueba
RF-01 Registrar usuarios	CU-01	CP-01, CP-02
RF-02 Modificar usuarios	CU-02	CP-03, CP-04
RF-03 Autenticar usuarios	CU-03	CP-05, CP-06, CP-07
RF-04 Eliminar documentos	CU-04	CP-08, CP-09

Para la tercera entrega, esta matriz permite relacionar directamente la SRS (Software Requirements Specification) de la primera entrega, los casos de uso de la segunda y las pruebas de la tercera.

Los CP deben relacionarse con RF o con CU?

1. Relación RF → CP

Se utiliza cuando diseñamos una prueba directamente a partir de un requerimiento.

Ejemplo:

RF-01: El sistema debe permitir registrar documentos.

CP-01: Verificar que el sistema permita registrar un documento con datos válidos.

Aquí comprobamos directamente el cumplimiento del requerimiento.

2. Relación CU → CP

Se utiliza cuando diseñamos pruebas a partir de los escenarios definidos en un caso de uso.

Ejemplo:

CU-01: Registrar documento.

El caso de uso establece:

- Flujo principal: el usuario ingresa datos válidos y el documento se registra correctamente.
- Flujo alternativo: el usuario omite un campo obligatorio.
- Flujo de excepción: ocurre un error al guardar el documento.

Podemos diseñar tres casos de prueba:

Caso de prueba	Escenario evaluado
CP-01	Registro exitoso de un documento.
CP-02	Registro con campos obligatorios vacíos.
CP-03	Error durante el almacenamiento.

Aquí verificamos los diferentes escenarios del caso de uso.

3. Relación RF → CU → CP

Esta es la relación más completa para trabajar la trazabilidad en el proyecto.

RF-01 – Registrar documentos

¿Qué debe hacer el sistema?

CU-01 – Registrar documento

¿Cómo interactúa el usuario con el sistema?

Casos de prueba

CP-01 – Registro con datos válidos

CP-02 – Campos obligatorios vacíos

CP-03 – Error de almacenamiento

Esto no significa que todos los casos de prueba deban diseñarse obligatoriamente a partir de un CU.

Por ejemplo, un requisito no funcional que establezca que las contraseñas deben almacenarse mediante un algoritmo de hash seguro puede verificarse mediante pruebas específicas sin necesitar un caso de uso independiente.

Recomendación

Utilizar una matriz de trazabilidad que incluya las tres relaciones cuando correspondan.

Requerimiento	Caso de uso	Caso de prueba
RF-01 Registrar documentos	CU-01 Registrar documento	CP-01 Registro correcto
RF-01 Registrar documentos	CU-01 Registrar documento	CP-02 Campos vacíos
RF-01 Registrar documentos	CU-01 Registrar documento	CP-03 Error de almacenamiento
RF-02 Buscar documentos	CU-02 Buscar documento	CP-04 Búsqueda existente
RF-02 Buscar documentos	CU-02 Buscar documento	CP-05 Documento inexistente
RNF-01 Protección de contraseñas	No aplica	CP-06 Verificación del almacenamiento seguro

Las relaciones no siempre son uno a uno. Un requerimiento funcional (RF) puede estar representado en varios casos de uso (CU), y un caso de uso puede satisfacer varios requerimientos funcionales. Asimismo, un RF o CU puede tener múltiples casos de prueba (CP) asociados, y un mismo CP puede verificar aspectos de más de un requerimiento.

Técnicas para el diseño de pruebas funcionales

Partición de equivalencia

La partición de equivalencia consiste en dividir los posibles datos de entrada en grupos o clases cuyos elementos deberían producir comportamientos semejantes.

Esta técnica permite seleccionar valores representativos sin necesidad de probar cada dato posible.

Ejemplo: Registro de edad

Un formulario permite ingresar edades entre 18 y 65 años, inclusive.

Se identifican tres particiones:

Partición	Datos	Resultado esperado
Inválida inferior	Menores de 18	Rechazado
Válida	Entre 18 y 65	Aceptado
Inválida superior	Mayores de 65	Rechazado

Se pueden seleccionar los valores 15, 30 y 70 como representantes de cada partición.

Análisis de valores límite

Los defectos suelen presentarse en los extremos de los rangos admitidos (también denominados “valores de borde o frontera”), por lo que resulta conveniente probar los valores situados en los límites y próximos a ellos.

Para el mismo ejemplo, se utilizarían:

- 17: inválido.
- 18: válido.
- 19: válido.
- 64: válido.
- 65: válido.
- 66: inválido.

La combinación de ambas técnicas permite elaborar pruebas más efectivas que la selección aleatoria de datos.

Tablas de decisión aplicadas a las pruebas

Las tablas de decisión permiten representar las distintas combinaciones de condiciones que afectan el comportamiento de un sistema y las acciones que deben ejecutarse en cada caso.

Son especialmente útiles cuando una funcionalidad depende de varias reglas de negocio.

En el diseño de pruebas, permiten identificar situaciones que podrían omitirse si se analizaran las condiciones individualmente.

Ejemplo: Autorización para eliminar documentos

Para que un documento pueda eliminarse, se establecen las siguientes condiciones:

- C1: el usuario está autenticado.
- C2: el usuario tiene permiso de eliminación.
- C3: el documento existe.

Condición o acción	Regla 1	Regla 2	Regla 3	Regla 4
Usuario autenticado	Sí	Sí	Sí	No
Tiene permiso	Sí	Sí	No	—
Documento existe	Sí	No	—	—
Eliminar documento	Sí	No	No	No
Mostrar error de autorización	No	No	Sí	Sí
Mostrar documento inexistente	No	Sí	No	No

El símbolo — significa que el valor de esa condición no modifica el resultado de la regla considerada.

Cada columna puede transformarse en uno o varios casos de prueba.

Por ejemplo, para la Regla 2 se debe ejecutar una prueba con un usuario autenticado, con permisos de eliminación, que intenta eliminar un documento inexistente.

El resultado esperado será un mensaje que informe que el documento no existe.

Las tablas de decisión también pueden utilizarse durante el análisis de requerimientos, pero en esta etapa se aplican específicamente al diseño de las pruebas.

En el ejemplo tenemos cuatro reglas:

- Regla 1: si el usuario está autenticado, tiene permisos y el documento existe, el sistema permite eliminar el documento.
- Regla 2: si el usuario está autenticado, tiene permisos, pero el documento no existe, el sistema informa que el documento no existe.
- Regla 3: si el usuario está autenticado, pero no tiene permisos, el sistema rechaza la operación por falta de autorización, independientemente de que el documento exista.
- Regla 4: si el usuario no está autenticado, el sistema rechaza la operación, independientemente de los permisos y de la existencia del documento.

El símbolo — significa que el valor de esa condición es irrelevante para la decisión (don't care).

Relación con los casos de prueba

Cada regla puede utilizarse como base para diseñar uno o varios casos de prueba.

Por ejemplo:

Caso de prueba CP-03 – Regla 3

- Precondición: usuario autenticado sin permisos de eliminación.
- Acción: intentar eliminar un documento.
- Resultado esperado: el sistema rechaza la operación y muestra un mensaje de autorización insuficiente.

En síntesis: una regla es una columna de la tabla que indica qué debe hacer el sistema ante una combinación determinada de condiciones.

Plan de testing

Un plan de testing, o plan de pruebas, es un documento que establece cómo se organizarán, ejecutarán y controlarán las pruebas del sistema.

De acuerdo con el enfoque de planificación y pruebas expuesto por Pressman, las actividades de prueba deben integrarse al proceso de desarrollo, definiendo responsabilidades, recursos, técnicas y criterios de evaluación.

Sommerville (2016) también destaca la necesidad de organizar las pruebas para obtener evidencia sistemática acerca del comportamiento y la confiabilidad del sistema.

Contenido propuesto para el plan de testing

1. Objetivo: establecer qué se pretende comprobar con las pruebas.

2. Alcance: identificar módulos y funcionalidades que serán evaluados.

3. Tipos de pruebas: especificar las pruebas unitarias, de integración, funcionales, de sistema y aceptación que correspondan.

4. Entorno: describir sistema operativo, navegador, base de datos y demás recursos utilizados.

5. Responsables: identificar a quienes diseñan, ejecutan y documentan las pruebas.

6. Casos de prueba: incorporar los casos seleccionados y su relación con los requerimientos.

7. Criterios de entrada: condiciones que deben cumplirse para comenzar las pruebas, por ejemplo, que el módulo esté implementado y el entorno disponible.

8. Criterios de salida: condiciones para dar por terminada la ejecución prevista, como ejecutar todos los casos prioritarios y resolver los defectos críticos.

9. Registro de incidentes: documentar los defectos encontrados y las acciones correctivas.

10. Informe de resultados: presentar las pruebas ejecutadas, aprobadas, fallidas y pendientes, incluyendo las limitaciones de la evaluación.

Ejemplo: Resultado de una ejecución

Indicador	Resultado
Casos planificados	20
Casos ejecutados	18
Casos aprobados	15
Casos fallidos	3
Casos pendientes	2
Porcentaje de aprobación sobre ejecutados	83,3 %

El porcentaje de aprobación se calcula mediante:

$$Aprobación = \frac{Pruebasaprobadas}{Pruebasejecutadas} \times 100$$

Este resultado no significa que el software tenga una calidad del 83,3 %. Únicamente indica la proporción de casos ejecutados que cumplieron sus resultados esperados.